

## سوال اول

در State verification ما حالت نهایی سیستم را بعد از اجرای برنامه ارزیابی می‌کنیم. مثلاً مقادیر بازگشتی سیستم را ارزیابی می‌کنیم، اما در Behavior verification ما علاقه‌مند به بررسی روند اجرای برنامه هستیم؛ یعنی چه متدهایی با چه ترتیبی و با چه مقادیری صدا زده شده‌اند. یعنی در روش اول output های SUT را ارزیابی می‌کنیم، اما در دومی Indirect output های SUT را ارزیابی می‌کنیم.

گاهی از اصطلاح‌های متفاوت برای اشاره به مفهومی واحد استفاده می‌شود. در این پاسخ ما بر اساس تعاریف [این مقاله](#) می‌توانیم بگوییم که برای State verification از Stub و برای Behavior verification از Mock استفاده می‌کنیم. البته این مقاله خیلی کوتاه اشاره می‌کند که می‌توان از Stub هم برای Behavior verification استفاده کرد اما بعضی در آن صورت به آن Spy می‌گویند، و در آخر گویی این سوال را به پاسخ گذاشته است. در نهایت با توجه به توضیحات کامل‌تر [این منبع](#) و اسلایدهای درس می‌توانیم به همان نتیجه اول برسیم که برای State verification از Stub و برای Behavior verification از Mock استفاده می‌کنیم.

## سوال دوم

Test Spy ها نوعی Test double هستند که رفتار Object واقعی را دارند مگر توابعی که ما آن‌ها را تغییر می‌دهیم تا اطلاعاتی از برنامه را در حین اجرا ذخیره کنند و برای Assertion استفاده کنیم. در واقع از آن برای Behavioral verification استفاده می‌کنیم.

ما از این نوع Test double برای تست کردن اطلاعاتی که در حالت عادی، برنامه بروز نمی‌دهد استفاده می‌کنیم، تا آن اطلاعات را در حین اجرا برنامه ذخیره کنیم. این اطلاعات می‌توانند شامل مقادیر آرگومان‌ها، مقادیر بازگشتی توابع صدا زده شده، نام توابعی که در طول اجرا صدا زده شده‌اند، تعداد بار صدا زده شدن یک تابع و ... باشند.

انواع Test Spy:

- **Retrieval Interface** - می‌توانیم یک Test Spy را به عنوان یک کلاس جداگانه با Retrieval Interface ای تعریف کنیم که اطلاعات ذخیره شده در حین اجرای برنامه را Expose کند. در تست در مرحله ساختن Fixture به جای استفاده از Object واقعی DOC، از این Test Spy استفاده می‌کنیم. بعد از اجرای SUT از آن Retrieval Interface استفاده می‌کنیم تا اطلاعات مورد نیازمان را بدست آوریم و آن‌ها را Assert کنیم.

- **Self Shunt** - می‌توانیم کلاس Testcase و Test Spy را ترکیب کنیم و به یک Object تبدیل کنیم که به نام Self Shunt شناخته می‌شود. در واقع متد تست خودش (this) را به عنوان DOC به SUT تزریق می‌کند. هر وقت که SUT متدی از DOC را صدا بزند، در واقع متدی از Object تست را صدا می‌زند، که در آن متد می‌تواند مقادیر واقعی را در متغیرهایی از آن Instance ذخیره کند در نتیجه متد تست به آن‌ها دسترسی دارد. دقت کنید که کلاس Testcase باید Interface آن DOC را پیاده‌سازی کند تا با متغیری از SUT که آن DOC را نگه می‌دارد type-compatible باشد.
- **Inner Test Double** - یک راه دیگر برای پیاده‌سازی Test Spy این است که آن را به صورت Anonymous inner class داخل متد تست پیاده‌سازی کنیم تا مقادیر واقعی را در متغیرهای آن Instance یا متغیرهای محلی آن متد تست ذخیره کنیم تا متد تست به آن‌ها دسترسی داشته باشد. این نوع در واقع روشی دیگر برای پیاده‌سازی Self Shunt هست.
- **Indirect Output Registry** - در این روش مقادیر واقعی پارامترهایی را که می‌خواهیم، در یک جا که متد تست ما دسترسی داشته باشد ذخیره می‌کنیم، به طور مثال می‌توانیم در یک فایل یا Registry object ذخیره کنیم. بعد از آن متد تست آن اطلاعات را می‌خواند و ارزیابی می‌کند.

## سوال سوم

(الف)

در کل ما ترجیح می‌دهیم که از Fresh fixture استفاده کنیم چون دیگر نگران این نیستیم که تست‌های قبلی چه تغییراتی در Fixture ایجاد کرده‌اند و اطمینان کامل از استتیت Fixture داریم، و حتی خیلی از موارد را نمی‌توانیم در Shared fixture تست کنیم زیرا باید دقیقاً در یک حالت خاص باشیم که چنین اطمینانی در Shared fixture معمولاً وجود ندارد. اما گاهی مجبور می‌شویم که از Shared fixture استفاده کنیم. کی؟ زمانی که Setup کردن Fixture خیلی هزینه بر باشد بهتر است از Shared fixture استفاده کنیم. مثلاً وقتی نیاز باشد که برای چندین تست از پایگاه داده استفاده کنیم؛ چون Setup کردن پایگاه داده بسیار زمان‌بر است. البته باید توجه داشته باشیم که همچنان باید مواظب باشیم که تست‌ها را به گونه‌ای بنویسیم که به تغییراتی که تست‌های قبلی در Fixture ایجاد کرده‌اند وابسته نباشند، مثلاً به جای ارزیابی مقدار مطلق یک متغیر، تفاوت قبل از اجرای تست و بعد از آن را ارزیابی کنیم.

(ب)

مزایا:

- **صرفه جویی در منابع** - هر تست فقط منابعی که نیاز دارد را ایجاد می‌کند، که باعث کاهش مصرف حافظه و زمان می‌شود.
- **اجرای سریع‌تر تست‌های اولیه** - تست‌های اولیه‌ای که به منابع زیادی نیاز ندارند، سریع‌تر اجرا می‌شوند چون نیازی به ایجاد آن منابع ندارند. در حالت خاص وقتی می‌خواهیم یک تست را به تنهایی اجرا کنیم اگر به بخشی از منابع نیاز داشته باشد سرعت اجرا بیش‌تر خواهد بود.
- **افزایش استقلال تست‌ها** - با ایجاد فقط منابع مورد نیاز هر تست، احتمال Leakage استتیت بین (حالت‌هایی وجود دارند که سرعت کاهش می‌یابد و در بخش معایب به آن اشاره خواهد شد).

تست‌ها کاهش می‌یابد. چون در روش Suite هر تست به همه‌ی منابع مورد نیاز برای تست‌ها دسترسی دارد، اما در این روش حداقل بعضی تست‌ها به منابع کمتری دسترسی دارند.

معایب:

- **پیچیدگی بیش‌تر کد و مدیریت سخت‌تر آن** - چون Setup کردن Fixture در تست‌های مختلف انجام می‌شود و یکپارچه نیستند کد را کمی پیچیده‌تر می‌کند. زیرا زمانی که همه‌ی Setup در کنار هم هستند، بهتر می‌توانیم بفهمیم Fixture چیست و آیا به درستی Setup می‌شود یا نه؛ وقتی که در کد تست باگی باشد، پیدا کردن آن دشوارتر خواهد شد.
- **کدهای تکراری** - اگر چند تست منابع مشابهی را نیاز داشته باشند باید اول همه‌ی آن‌ها چک کنیم که اگر آن منبع Setup نشده آن را Setup کنند. و این کد تکراری خواهد بود در حالی که در روش Suite کد Setup تمام Fixture فقط یکبار در تابع اولیه که معمولاً با @BeforeAll نشان داده می‌شود نوشته می‌شود.
- **افزایش زمان اجرا در بعضی مواقع** - فرض کنید که یک تست برای Setup کردن Fixture باید دیتاهایی را از پایگاه داده بخواند. همچنین بخشی از این دیتا را تست قبلی هم نیاز داشته و آن بخش در تست قبل خوانده شده است. و در این تست مابقی دیتای مورد نیاز خوانده می‌شود. یعنی خواندن دیتا از پایگاه داده در دو قسمت انجام شده است و می‌دانیم که بهتر است دیتا را از پایگاه داده به صورت یک‌جا بخوانیم و تقسیم کردن آن به دو بخش مجزا زمان‌برتری را افزایش می‌دهد.

(ج)

دو روش را بیان می‌کنیم که روش دوم با خواسته پرسش سازگارتر است:

- **رویکرد تغییر روش ارزیابی** - در بخش ارزیابی تست‌ها به جای ارزیابی مقدار مطلق یک متغیر، تفاوت مقدار متغیر قبل از اجرای تست و بعد از آن را ارزیابی کنیم. در این صورت می‌توانیم از تغییرات Fixture در تست‌های قبل مستقل شویم.
- **رویکرد مدیریت Fixture** - می‌توانیم هر تست را موظف کنیم که تغییری در Fixture ایجاد کرد در پایان کار یعنی در بخش Tear Down آن را به حالت قبلی خود بازگرداند. اما همیشه Revert کردن تغییرات به این سادگی نخواهد بود مثلاً در سیستم بانکی اگر پولی جابه‌جا شد نمی‌توانیم بگوییم برعکس آن جابه‌جایی را انجام می‌دهیم، زیرا در این بین Log هایی در سیستم ثبت شده است و باید آن‌ها را هم حذف کنیم. علاوه بر این در بعضی مواقع ممکن است که کلاً Revert کردن غیر ممکن باشد.